Self-Learning Plan: React Native + Expo in 4 Weeks

Self-paced, no teacher · level: easy to medium · 4 weeks, 4 tasks a week.

There is no day-by-day timetable. Each week has a goal, and you plan your own days. Every week has two parts:

1. **What to learn this week.** The concepts the week's tasks need. Each concept has an article or documentation page to read, and a file in this app where you can see it in real code.
2. **Tasks this week.** 4 tasks that add real features to this app. Each task says its **type**, its **objective**, **what the user will see**, **which files to touch**, **the exact steps** and **how to know it's done**.

Learn first, then do the tasks. A task lists the concepts it uses, so you can go back to them.

Task types

Type	What it means	You mostly practise
UI	Build or change what the user **sees**: layout, components, colors, dark mode. Little logic.	Components, props, Flexbox, NativeWind, theme
Functional	Make the app **do** something: load or change data, validate input, store values, write tests.	API, React Query, zod, forms, tests, native APIs
UI + Functional	A complete feature: a new screen or control **and** the data or behaviour behind it.	Both, plus navigation

Size tells you how big a task is: **S** (a few hours) · **M** (about a day) · **L** (1–2 days).

The 4 weeks at a glance

Week	Theme	Goal of the week	Tasks
1	Screens and UI	Build screens with the app's components, theme colors and routes	1.1 Support row (UI + Functional) · 1.2 Rating stars (UI) · 1.3 Details card (UI) · 1.4 Reviews screen (UI + Functional)
2	Data: API, React Query, forms	Read and change server data, validate forms, write tests	2.1 Sort by price (Functional) · 2.2 Out-of-stock action (Functional) · 2.3 Category screen (UI + Functional) · 2.4 Discount field (Functional)
3	Build your own module	Create a full `users` module, and log in	3.1 Users data layer (Functional) · 3.2 Users tab (UI) · 3.3 User profile (UI + Functional) · 3.4 Login (UI + Functional)
4	Polish and a mini project	Offline support, cross-module widgets, and a cart built on your own	4.1 Offline banner (UI + Functional) · 4.2 Team widget (UI) · 4.3 Cart tab (UI + Functional) · 4.4 Cart summary (UI)

Totals: 5 UI · 5 Functional · 6 UI + Functional.

How to learn and how to work

- **Learning:** read the links for each concept. You don't need to memorise anything. Then open the "See it in the app" file and find the concept in real code. Answer the "Check yourself" question in your own words in `PROGRESS.md`.
- **Tasks:** one branch and one pull request (PR) per task, titled like `Task 2.2 (Functional): out-of-stock action`. Run `npm run validate` before pushing.
- **Every UI task:** check it in **light and dark mode** (Settings → Theme) and on a small phone. Use theme colors only (see the cheat sheet), never hex colors.
- **Every Functional task:** check the **error case** too (turn on airplane mode, or type invalid input), not only the happy path.
- **When you're stuck:** re-read the task, open the file it points to (the pattern usually already exists), search the exact error message, and re-read the concept's links. If you use an AI assistant, ask it to **explain**, not to write the code. Still stuck after 45 minutes? Write the blocker in your PR and move on.

Setup (before Week 1)

- [ ] Install **Node.js** (LTS), **Git**, **VS Code** (ESLint, Prettier, Tailwind CSS IntelliSense extensions) and **Expo Go** on your phone (SDK 58).
- [ ] Create a **GitHub** account and **fork** this repository (how to fork).
- [ ] Clone your fork, `npm install`, `npm start`, scan the QR code with your phone (GETTING-STARTED.md).
- [ ] Learn how to open a PR from your fork (guide).
- [ ] Create `PROGRESS.md` in your fork (template at the end).

Cheat sheet: building screens in this app

See every component live in the app: **Settings** → **UI Sandbox**.

You need	Use
Text	`Text` from `@/shared/ui/text` (`variant="h3"` , `"large"` , `"muted"`)
Button	`Button` from `@/shared/ui/button` (`variant="outline"` , `"destructive"`)
Card / box	`className="rounded-lg border border-border bg-card p-4"`
Badge, Avatar, Input	`@/shared/ui/badge` · `@/shared/ui/avatar` · `@/shared/ui/input`
Icon	`Icon` from `@/shared/ui/icon` + an icon from lucide
Loading/error/empty	`LoadingState` , `ErrorState` , `EmptyState` from `@/shared/components/screen-states`

Colors: never hex (`#4f46e5`). Use theme classes so dark mode works by itself: `bg-background` · `bg-card` · `text-foreground` · `text-muted-foreground` · `text-primary` · `bg-primary` · `text-destructive` · `bg-success` · `border-border` .

Building…	Copy from
A list screen	`src/modules/catalog/features/product-list/product-list-screen.tsx`
A details screen	`src/modules/catalog/features/product-detail/product-detail-screen.tsx`
A form	`src/modules/catalog/components/product-form.tsx`
A settings-style list	`src/modules/settings/features/app-settings/settings-screen.tsx`

Week 1: Screens and UI

Goal of the week: build screens with the app's components, theme colors and routes. By the end you can add a new screen, put UI on it and navigate to it.

What to learn this week

Concept 1.1 — How this project is organised

- What to learn: what `src/app`, `src/modules` and `src/shared` are; what a module and a feature are; where a screen's code lives.
- Read: README ("What the app does", "Project structure") · Feature reference ("What is a feature?")
- See it in the app: `src/app/(tabs)/products.tsx` → `src/modules/catalog/features/product-list/`
- Check yourself: if you add a new screen about products, which folder does its code go in?

Concept 1.2 — React components, props and lists

- What to learn: a component is a function that returns UI; props pass data in; `map` renders a list; every list item needs a `key`; `condition ? a : b` shows something conditionally.
- Read: Your first component · Passing props · Rendering lists · Conditional rendering
- See it in the app: `src/modules/catalog/features/product-list/product-card.tsx`
- Check yourself: what are the props of `ProductCard`?

Concept 1.3 — TypeScript for components

- What to learn: basic types (`string`, `number`, `boolean`), object types, optional props (`name?: string`), typing a component's props.
- Read: Everyday types · React + TypeScript cheatsheet ("Basic")
- See it in the app: the props type of `ProductCard` in `product-card.tsx`
- Check yourself: how do you make a prop optional?

Concept 1.4 — React Native core components

- What to learn: `View` (a box), `Text` (all text must be inside it), `Pressable` (tappable), `ScrollView`, `Image`; opening other apps with `Linking` (email, phone).
- Read: Core components · Pressable · Linking
- See it in the app: `src/shared/components/settings-row.tsx`
- Check yourself: why can't you put a plain string directly inside a `View`?

Concept 1.5 — Layout and styling with theme colors

- What to learn: Flexbox (`flex-row`, `justify-between`, `items-center`, `gap-3`); NativeWind `className`; theme tokens instead of hex colors; dark mode.
- Read: React Native: Flexbox · NativeWind docs · THEMING.md ("The tokens")
- See it in the app: the price row in `product-detail-screen.tsx`
- Check yourself: which class puts two items on one line, one left and one right?

Concept 1.6 — Data shapes with zod

- What to learn: a schema describes the data the API sends; `z.string()`, `z.number()`, `.optional()`, `.default()`, `z.array()`; the TypeScript type comes from the schema.
- Read: Zod: basic usage · try `https://dummyjson.com/products/1` in your browser
- See it in the app: `src/modules/catalog/data/product.schema.ts`
- Check yourself: what happens if the API sends a field that isn't in the schema?

Concept 1.7 — Navigation with Expo Router

- What to learn: every file in `src/app` is a screen; folders become URLs; `[id]` is a dynamic part of the URL; `router.push(...)` opens a screen; the Stack in `_layout.tsx`.
- Read: Expo Router: core concepts · Navigating between pages · ARCHITECTURE.md ("Navigation")
- See it in the app: `src/app/products/[id]/index.tsx` and `src/modules/catalog/catalog.routes.ts`
- Check yourself: which file is shown for the URL `/products/5/edit`?

Tasks this week

Task	Name	Type	Size	Uses concepts
1.1	"Contact support" row	UI + Functional	S	1.1, 1.4
1.2	Rating stars component	UI	S	1.2, 1.3
1.3	"Details" card	UI	M	1.5, 1.6
1.4	Reviews screen	UI + Functional	L	1.1, 1.6, 1.7

Task 1.1 — "Contact support" row in Settings

- **Type:** UI + Functional · **Size:** S
- **Objective:** let users email the support team directly from the app.
- **What the user sees:** in Settings, a new row with a mail icon, the label "Contact support" and the description "Email our support team". Tapping it opens the phone's email app with `support@example.com` already in the "To" field and "Help with the app" as the subject.
- **Files:** `src/modules/settings/features/app-settings/settings-screen.tsx` (change).
- **What to do:**
  1. Open `settings-screen.tsx` and find the "UI Sandbox" `SettingsRow` in the "Developer" group. Read its props: `icon`, `label`, `description`, `onPress`.
  2. Add a new `SettingsGroup` titled "Help" (above "Developer"), with a new `SettingsRow` in it.
  3. Import the `Mail` icon from `lucide-react-native` and pass it as `icon`. Set `label` and `description` as above.
  4. Import `Linking` from `react-native`. In `onPress`, call `Linking.openURL('mailto:support@example.com?subject=Help%20with%20the%20app')`.
  5. Put the email address in a constant at the top of the file (`const SUPPORT_EMAIL = '…'`), not inline.
- **Done when:**
  - ☐ The row looks like the other rows in light and dark mode.
  - ☐ Tapping it opens the email app with the address and subject filled in.
  - ☐ `npm run validate` passes.

Task 1.2 — Rating stars component

- **Type:** UI · **Size:** S
- **Objective:** show product ratings as 5 stars, the way shopping apps do, with one reusable component.
- **What the user sees:** on every product card, 5 small stars instead of the single star. A product rated 4.6 shows 5 filled stars; 3.2 shows 3 filled and 2 empty; the number "4.6" can appear after the stars.
- **Files:** `src/modules/catalog/components/rating.tsx` (new) · `src/modules/catalog/features/product-list/product-card.tsx` (change) · `src/modules/catalog/components/catalog-showcase.tsx` (change).
- **What to do:**
  1. Create `rating.tsx` exporting `Rating` with props `value: number`, `showValue?: boolean` and `size?: 'sm' | 'md'` (default `'sm'`).
  2. Render 5 stars with `Array.from({ length: 5 }, (_, i) => …)`. Star number `i + 1` is **filled** when `i + 1 <= Math.round(value)`. Give each star a `key`.
  3. Filled star: copy the classes from the star in `product-card.tsx` (`fill-amber-400 text-amber-400`). Empty star: `text-muted-foreground` with no fill.
  4. When `showValue` is true, show `value.toFixed(1)` after the stars with `variant="muted"`.
  5. Give the whole row `accessibilityLabel={` Rated ${value.toFixed(1)} out of 5 `}` so screen readers say one sentence, not "star star star".
  6. In `product-card.tsx`, replace the single star + number with `<Rating value={product.rating} showValue />`.
  7. Add 3 examples (1.0, 3.2, 4.6) to `catalog-showcase.tsx` so the component is visible in the UI Sandbox.
- **Done when:**
  - ☐ Product cards show the right number of filled stars.
  - ☐ Empty stars are visible in both light and dark mode.
  - ☐ The examples show in Settings → UI Sandbox.

Task 1.3 — "Details" card on the product screen

- **Type:** UI · **Size:** M
- **Objective:** show customers the warranty, shipping and return information before they buy.
- **What the user sees:** under the product description, a card titled "Details" with three rows: **Warranty**, **Shipping**, **Returns**. Each row has the label on the left (grey) and the value on the right. If the API has no value for a row, that row is hidden. If all three are missing, the whole card is hidden.
- **Files:** `src/modules/catalog/data/product.schema.ts` (change) · `src/modules/catalog/features/product-detail/product-detail-screen.tsx` (change).
- **What to do:**
  1. Open `https://dummyjson.com/products/1` and find `warrantyInformation`, `shippingInformation` and `returnPolicy`.
  2. Add the three fields to `productSchema` as `z.string().optional()`.
  3. In `product-detail-screen.tsx`, build a list of rows: `[{ label: 'Warranty', value: product.warrantyInformation }, …].filter((row) => row.value)`.
  4. If the list is not empty, render a card (`rounded-lg border border-border bg-card p-4`) with a title `Text` ("Details", `font-semibold`) and the rows.
  5. Each row: `flex-row justify-between gap-4 py-2`; label `text-muted-foreground`; value `text-foreground text-right flex-1`. Put a `Separator` between rows (not after the last).

- **Done when:**
  - [ ] Product 1 shows all three rows; long values wrap without pushing the label off screen.
  - [ ] Card colors are correct in dark mode.
  - [ ] `npm run validate` passes.

Task 1.4 — Reviews screen

- **Type:** UI + Functional · **Size:** L
- **Objective:** let customers read what other buyers said about a product.
- **What the user sees:** on the product screen, a "See all reviews (3)" button. It opens a new screen titled "Reviews" that lists every review: reviewer name (bold), stars (your `Rating`), the comment and the date. The back button returns to the product. A product with no reviews shows "No reviews yet".
- **Files:** `product.schema.ts` (change) · `src/modules/catalog/features/product-reviews/` (new: `product-reviews-screen.tsx`, `index.ts`, `README.md`) · `src/modules/catalog/index.ts` · `src/modules/catalog/catalog.routes.ts` · `src/app/products/[id]/reviews.tsx` (new) · `src/app/_layout.tsx` · `product-detail-screen.tsx`.
- **What to do:**
  1. **Data:** add `reviewSchema` (`rating: z.number()`, `comment: z.string()`, `reviewerName: z.string()`, `date: z.string()`) and add `reviews: z.array(reviewSchema).default([])` to `productSchema`.
  2. **Screen:** create `product-reviews-screen.tsx`. It takes `productId: number` as a prop and calls `useProduct(productId)`. Show `LoadingState` while loading, `ErrorState` on error, `EmptyState` ("No reviews yet") when `reviews` is empty, otherwise a `ScrollView` of review cards.
  3. **Public API:** export the screen from the feature's `index.ts`, then from `src/modules/catalog/index.ts`.
  4. **Route builder:** add `reviews: (id: number) => …` to `catalogRoutes` (look at how `edit` is built).
  5. **Route file:** create `src/app/products/[id]/reviews.tsx` by copying `src/app/products/[id]/edit.tsx`, and render your screen. Register it in `src/app/_layout.tsx` with the title "Reviews".
  6. **Button:** on the product screen, add an outline `Button` "See all reviews (n)" that calls `router.push(catalogRoutes.reviews(product.id))`.
  7. Write a short `README.md` for the feature (copy the shape of the product-list README).
- **Done when:**
  - [ ] Any product's reviews open, show stars and comments, and Back works.
  - [ ] Loading, error and empty states all show correctly.
  - [ ] `npm run lint` passes (no deep imports from `src/app`).

End-of-week check

- [ ] 4 PRs open · `PROGRESS.md` updated · every screen checked in dark mode.

Week 2: Data — API, React Query and forms

Goal of the week: read and change server data the way this app does it, validate forms and write your first tests.

What to learn this week

Concept 2.1 — How an app talks to a server

- What to learn: HTTP requests (GET to read, POST/PUT/DELETE to change), status codes (200, 404, 500), JSON.
- Read: MDN: An overview of HTTP · MDN: HTTP status codes · DummyJSON products API
- See it in the app: `src/shared/api/http-client.ts` (the only place that calls `fetch`)
- Check yourself: what does a 404 status mean?

Concept 2.2 — The layers: screen → hook → service → HTTP client

- What to learn: screens never call the API directly; a **service** calls the API and checks the answer with zod; a **hook** gives screens the data plus loading and error state.
- Read: ARCHITECTURE.md ("The layers inside a slice" and "READ: the list")
- See it in the app: `products.service.ts` → `product.queries.ts` → `product-list-screen.tsx`
- Check yourself: why does the service validate the response with zod?

Concept 2.3 — Reading data with React Query

- What to learn: `useQuery` gives `data`, `isPending`, `isError`, `refetch`; data is cached, so going back to a screen is instant; every screen shows loading, error, empty or data.
- Read: TanStack Query: overview · Queries · TkDodo: Practical React Query
- See it in the app: `useProduct` in `src/modules/catalog/data/product.queries.ts`
- Check yourself: what does the screen show while `isPending` is true?

Concept 2.4 — Query keys

- What to learn: the key is the cache "address" of the data; every value that changes the result (search, sort, id) must be part of the key.
- Read: Query keys · TkDodo: Effective React Query Keys
- See it in the app: `src/modules/catalog/data/product.keys.ts`
- Check yourself: what goes wrong if the search text is not in the key?

Concept 2.5 — Changing data with mutations

- What to learn: `useMutation` sends a POST/PUT/DELETE; after success, the cache is updated so every screen shows the new data.
- Read: Mutations · Updates from mutation responses
- See it in the app: `src/modules/catalog/features/edit-product/use-update-product.ts`
- Check yourself: after editing a product, why does the list update without a refresh?

Concept 2.6 — Forms with validation

- What to learn: react-hook-form holds the form's values; a zod schema checks them; errors show under each field.
- Read: React Hook Form: get started · useForm
- See it in the app: `src/modules/catalog/components/product-form.tsx` and `productFormSchema`
- Check yourself: where is the rule "Price must be greater than 0" written?

Concept 2.7 — Unit tests

- What to learn: a test runs a function and checks the result with `expect`; a "fake fetch" pretends to be the server.
- Read: Jest: getting started · expect
- See it in the app: `src/modules/catalog/data/__tests__/product.schema.test.ts`
- Check yourself: run `npm test`. How many tests pass?

Tasks this week

Task	Name	Type	Size	Uses concepts
2.1	Sort by price	Functional	M	2.3, 2.4
2.2	"Mark out of stock" action	Functional	S	2.5
2.3	Products by category	UI + Functional	L	2.1, 2.2, 2.3, 2.7
2.4	"Discount %" form field	Functional	M	2.5, 2.6, 2.7

Task 2.1 — Sort by price

- **Type:** Functional · **Size:** M
- **Objective:** let shoppers find cheap or premium products quickly.
- **What the user sees:** above the product list, a small outline button showing the current sort: "Sort: Default". Each tap cycles to "Sort: Price ↑" (low to high), then "Sort: Price ↓" (high to low), then back to "Default". The list re-orders after each tap, and infinite scroll still loads more products in the same order.
- **Files:** `src/modules/catalog/features/product-list/product-list-screen.tsx` · `src/modules/catalog/data/product.queries.ts` · `src/modules/catalog/data/product.keys.ts`.
- **What to do:**
  1. Read `productsService.list` in `products.service.ts`: it already accepts `sortBy` and `order`. Try `https://dummyjson.com/products?sortBy=price&order=asc` in your browser.
  2. Change `productQueries.list(search)` to take `{ search, sortBy, order }` and pass them to `productsService.list(...)`.
  3. Add `sortBy` and `order` to the key: `productKeys.list({ search, sortBy, order })` (update the type in `product.keys.ts`). This makes each sort its own cache entry.
  4. Change `useProductList` to take the same object, and fix every caller (TypeScript will show you where).
  5. In `product-list-screen.tsx`, keep the sort in `useState`, show the button, and pass the sort into `useProductList`.
- **Done when:**
  - ☐ Each tap re-orders the list; scrolling down keeps the order.
  - ☐ Searching while sorted still works.

- [ ] Switching back to a sort you already used is instant (it comes from the cache).

Task 2.2 — "Mark out of stock" action

- **Type:** Functional · **Size:** S
- **Objective:** let a store manager mark a product as sold out in one tap.
- **What the user sees:** long-press a product (or tap ":") and the actions sheet shows a new item "Mark out of stock" under "Edit product". After tapping it, the sheet closes and the product's badge says "Out of stock" right away, in the list and on the detail screen. If the request fails, an alert explains the error.
- **Files:** `src/modules/catalog/features/product-list/product-actions-sheet.tsx` · `src/modules/catalog/features/product-list/product-list-screen.tsx`.
- **What to do:**
  1. In `product-actions-sheet.tsx`, add a prop `onMarkOutOfStock: (product: Product) => void` and a new `ActionRow` with the `PackageX` icon. Hide it when `product.stock === 0`.
  2. In `product-list-screen.tsx`, import `useUpdateProduct` from `'../edit-product'` (a sibling feature's public API) and create the mutation.
  3. Handle the new prop: dismiss the sheet, then call `updateProduct.mutate({ id: product.id, payload: productFormSchema.parse({ ...productToFormInput(product), stock: '0' }) })`.
  4. Add `onError` that shows `Alert.alert('Update failed', getErrorMessage(error))` (copy how delete handles errors in the same screen).
- **Done when:**
  - [ ] The badge changes to "Out of stock" without a refresh, in the list and on the detail screen.
  - [ ] The item is hidden for products that are already out of stock.
  - [ ] In airplane mode, you see an error alert instead of nothing.

Task 2.3 — Products by category screen

- **Type:** UI + Functional · **Size:** L
- **Objective:** let shoppers see all products of one category (for example all smartphones).
- **What the user sees:** on a product's detail screen, the category badge ("Smartphones") is now tappable. It opens a screen titled with the category name that lists all its products, using the same cards as the Products tab. Tapping a card opens its details.
- **Files:** everything listed in ADDING-A-FEATURE.md (endpoint, key, service, hook, `features/products-by-category/`, route builder, route file, README) · `product-detail-screen.tsx` · `src/modules/catalog/data/__tests__/products.service.test.ts`.
- **What to do:**
  1. Follow ADDING-A-FEATURE.md steps 1–9. It builds exactly this screen and includes the code. Type it yourself instead of copy-pasting, and read each comment.
  2. Add a test for the new service function in `products.service.test.ts`: one test that the right URL is called and the products come back, one that a 404 throws.
  3. On `product-detail-screen.tsx`, wrap the category `Badge` in a `Pressable` with `accessibilityRole="link"` that calls `router.push(catalogRoutes.byCategory(product.category))`.
- **Done when:**
  - [ ] Tapping "Smartphones" on a phone's detail page lists all smartphones.
  - [ ] The screen has loading, error and empty states.

- [ ] `npm test` passes, including your two new tests.

Task 2.4 — "Discount %" field in the product form

- **Type:** Functional · **Size:** M
- **Objective:** let a store manager set a discount when creating or editing a product, with validation.
- **What the user sees:** a "Discount (%)" input under "Stock" in the create and edit forms. It opens a number keypad and starts at `0` for a new product. Typing `95` shows "Discount must be between 0 and 90" under the field and blocks saving; typing `20` saves, and the product then shows a 20% discount badge and the reduced price.
- **Files:** `src/modules/catalog/data/product.schema.ts` · `src/modules/catalog/components/product-form.tsx` · `src/modules/catalog/data/__tests__/product.schema.test.ts`.
- **What to do:**
  1. In `productFormSchema`, add `discountPercentage`. Copy how `stock` uses `numberFromText` (it already rejects empty, non-numbers and negatives), then add `.refine((v) => v <= 90, 'Discount must be between 0 and 90')`.
  2. Add the field to `productToFormInput` (`String(product.discountPercentage)`) and to `emptyProductForm` (`'0'`).
  3. In `product-form.tsx`, copy the "Stock" field block, change the name and label, and keep `keyboardType="number-pad"`.
  4. In `product.schema.test.ts`, add 3 tests: `'20'` is accepted as `20`, `'95'` is rejected, `'-5'` is rejected.
- **Done when:**
  - [ ] `95` shows the error and doesn't save; `20` saves and the discount shows on the product.
  - [ ] Editing a product shows its current discount in the field.
  - [ ] `npm test` passes.

End-of-week check

- [ ] 4 PRs open · `PROGRESS.md` updated · `npm test` passes.

Week 3: Build your own module

Goal of the week: create a complete `users` module on your own by following the recipe, and add login.

What to learn this week

Concept 3.1 — Modules and their public API

- What to learn: a module is a business area (`catalog` , `users`); other code may only import its `index.ts` ; ESLint blocks "deep" imports.
- Read: Module reference · ARCHITECTURE.md ("Module vs feature", "The import rules") · Jimmy Bogard: Vertical Slice Architecture
- See it in the app: `src/modules/catalog/index.ts`
- Check yourself: why can't the Home module import `@/modules/catalog/data/products.service` ?

Concept 3.2 — Building a module, step by step

- What to learn: the order: endpoints → schema → service → query keys → queries → features → public API → routes.
- Read: ADDING-A-MODULE.md (the whole recipe) · DummyJSON users API
- Check yourself: which file is the only one that contains API paths?

Concept 3.3 — Testing a service with a fake fetch

- What to learn: replace `fetch` with a Jest mock, return a fake answer, check the result or the error.
- Read: Jest: mock functions · Jest: testing async code
- See it in the app: `src/modules/catalog/data/__tests__/products.service.test.ts`
- Check yourself: what does `mockFetchOnce(body, 404)` simulate?

Concept 3.4 — Lists and tabs

- What to learn: `FlashList` for long lists (`data` , `renderItem` , `keyExtractor`); adding a tab in `(tabs)/_layout.tsx` .
- Read: FlashList: usage · Expo Router: tabs
- See it in the app: `src/app/(tabs)/_layout.tsx`
- Check yourself: what does `keyExtractor` do?

Concept 3.5 — Dynamic routes and params

- What to learn: a route like `users/[id].tsx` reads `id` with `useLocalSearchParams` and passes it to the screen as a prop.
- Read: Expo Router: URL parameters
- See it in the app: `src/app/products/[id]/index.tsx`
- Check yourself: why does the route pass `productId` as a prop instead of the screen reading the URL itself?

Concept 3.6 — Login and storing a token safely

- What to learn: login sends username and password and gets back an **access token**; the token proves who you are on later requests; it must be stored securely, not in plain storage.
- Read: DummyJSON auth · Expo SecureStore
- Check yourself: why must a token never go in an `EXPO_PUBLIC_*` variable or plain storage?

Tasks this week

Task	Name	Type	Size	Uses concepts
3.1	Users data layer (+ tests)	Functional	M	3.1, 3.2, 3.3
3.2	Users tab	UI	M	3.4
3.3	User profile screen	UI + Functional	M	3.5, 1.4
3.4	Login	UI + Functional	L	2.6, 3.6

Task 3.1 — Users data layer (with tests)

- **Type:** Functional · **Size:** M
- **Objective:** give the app everything it needs to load users from the API, tested, before building any screen.
- **What the user sees:** nothing yet. This task is the foundation for Tasks 3.2, 3.3 and 4.2.
- **Files (all new, in `src/modules/users/`):** `data/users.endpoints.ts` · `data/user.schema.ts` · `data/users.service.ts` · `data/user.keys.ts` · `data/user.queries.ts` · `data/__tests__/users.service.test.ts` · `index.ts` · `README.md`.
- **What to do:**
  1. Open `https://dummyjson.com/users?limit=2` and `https://dummyjson.com/users/1` and look at the fields you'll need: `id`, `firstName`, `lastName`, `email`, `phone`, `image`, `address.city`, `company.title`.
  2. Follow ADDING-A-MODULE.md steps 1–5 (endpoints, schema, service, keys, read hooks) and step 8 (public API). You should end up with `useUserList()` and `useUser(id)`.
  3. Copy `products.service.test.ts` to `users.service.test.ts` and write 3 tests: the list loads and is parsed, one user loads, a 404 throws an error.
  4. Write `README.md` for the module (copy the shape of the catalog README: purpose, folders, public API).
- **Done when:**
  - ☐ `npm run typecheck` and `npm test` pass.
  - ☐ `index.ts` exports only hooks and types, never the service.

Task 3.2 — Users tab

- **Type:** UI · **Size:** M
- **Objective:** show a list of everyone on the team.
- **What the user sees:** a new "Users" tab (people icon) between Products and Settings. It shows a scrolling list; each row has a round photo, the full name (bold) and the email (grey) under it. While loading there's a spinner; on error, a message with a "Try again" button.
- **Files:** `src/modules/users/features/user-list/` (new: `user-list-screen.tsx`, `user-row.tsx`, `index.ts`, `README.md`) · `src/modules/users/index.ts` · `src/app/(tabs)/users.tsx` (new) ·

`src/app/(tabs)/_layout.tsx`.

- **What to do:**
  1. Create `user-row.tsx`: a `Pressable` row with `Avatar` (photo, initials as fallback), name (`font-medium`) and email (`text-muted-foreground`). Wrap it in `memo` like `ProductCard`.
  2. Create `user-list-screen.tsx`: copy `product-list-screen.tsx` and delete what you don't need (search, sheets, delete, floating button). Use `useUserList()` and a `FlashList` with `keyExtractor={(user) => String(user.id)}`.
  3. Keep `LoadingState` and `ErrorState` (with `onRetry={refetch}`).
  4. Export the screen from the module's `index.ts`. Create `src/app/(tabs)/users.tsx` that only renders it.
  5. In `(tabs)/_layout.tsx`, add a `Tabs.Screen` named `users` with the title "Users" and the `Users` icon from lucide.
- **Done when:**
  - ☐ The Users tab scrolls smoothly through all users.
  - ☐ Loading and error states show (test the error with airplane mode).
  - ☐ Rows look right in dark mode.

Task 3.3 — User profile screen

- **Type:** UI + Functional · **Size:** M
- **Objective:** let people see a team member's details and contact them in one tap.
- **What the user sees:** tapping a user in the Users tab opens a profile screen: a big round photo at the top, the full name (`h3`), job title (grey), then a card with Email, Phone and City rows. Under the card, two buttons side by side: **Call** (opens the phone dialer with the number) and **Email** (opens the email app).
- **Files:** `src/modules/users/features/user-detail/` (new) · `src/modules/users/users.routes.ts` (new) · `src/modules/users/index.ts` · `src/app/users/[id].tsx` (new) · `src/app/_layout.tsx` · `user-list-screen.tsx`.
- **What to do:**
  1. Create `users.routes.ts` with `usersRoutes.detail(id)` (copy `catalog.routes.ts`).
  2. Create `user-detail-screen.tsx` that takes `userId: number` and calls `useUser(userId)`. Handle loading and error states.
  3. Build the layout with a `ScrollView`, an `Avatar` (`size-24`), the info card (reuse the row style from Task 1.3) and the two buttons in a `flex-row gap-3`.
  4. Call: `Linking.openURL(` tel:${user.phone} `)`. Email: `Linking.openURL(` mailto:${user.email} `)`.
  5. Create `src/app/users/[id].tsx` (copy `src/app/products/[id]/index.tsx`), read `id` with `useLocalSearchParams`, pass it as `userId`. Register it in `src/app/_layout.tsx` with the title "Profile".
  6. In the Users tab, make each row open `usersRoutes.detail(user.id)`.
- **Done when:**
  - ☐ Tapping a user opens their profile; Back returns to the list.
  - ☐ Call opens the dialer with the number; Email opens the email app.

Task 3.4 — Login

- **Type:** UI + Functional · **Size:** L
- **Objective:** let a user sign in and stay signed in after closing the app.

- **What the user sees:** in Settings, a "Sign in" row. It opens a Login screen with Username and Password inputs (password hidden) and a "Sign in" button. Wrong details show "Invalid username or password" under the form; the button shows a loading state while signing in. After signing in, the screen closes and Settings shows "Signed in as Emily Johnson" and a "Log out" row. Closing and reopening the app keeps you signed in.
- **Files:** `src/modules/auth/` (new: `data/auth.endpoints.ts`, `data/auth.schema.ts`, `data/auth.service.ts`, `data/token-storage.ts`, `features/login/`, `index.ts`, `README.md`) · `src/app/login.tsx` (new) · `src/app/_layout.tsx` · `settings-screen.tsx`.
- **What to do:**
    1. Install SecureStore: `npx expo install expo-secure-store`.
    2. **Data:** endpoint `POST /auth/login`; a schema for the response (`accessToken`, `firstName`, `lastName`, `username`, `image`); a service function `login(username, password)` that validates the response with `parseResponse`.
    3. **Token storage:** in `token-storage.ts`, three functions using SecureStore: `saveSession`, `readSession`, `clearSession`. Store the token and the user's name.
    4. **Hooks:** `useSession()` (a query that reads the stored session), `useLogin()` (a mutation that calls the service, saves the session, then invalidates `useSession`), and `useLogout()`.
    5. **Login screen:** a form with react-hook-form + a zod schema (both fields required), like `product-form.tsx`. Test user: `emilys` / `emilyspass`. On success, `router.back()`.
    6. **Settings:** if `useSession()` has a user, show "Signed in as …" and a "Log out" row; otherwise a "Sign in" row that opens `/login`. Settings imports only from `@/modules/auth`.
- **Done when:**
    - ☐ Wrong password shows the error; right password signs in.
    - ☐ Close and reopen the app: still signed in. Log out works.
    - ☐ The token is never logged or shown on screen. `npm run validate` passes.

End-of-week check

- ☐ 4 PRs open · `PROGRESS.md` updated · `npm run lint` passes (no deep imports).

Week 4: Polish and a mini project

Goal of the week: make the app handle being offline, connect modules through widgets, and build a cart feature mostly on your own.

What to learn this week

Concept 4.1 — Online and offline

- What to learn: the app can know when the phone is offline; React Query pauses requests and resumes them when the connection is back.
- Read: expo-network · TanStack: onlineManager
- See it in the app: `src/shared/lib/query-client.ts`
- Check yourself: what happens to a request started while offline?

Concept 4.2 — Shared components and the root layout

- What to learn: `src/shared/components` holds UI used everywhere; `src/app/_layout.tsx` wraps every screen, so something placed there shows on all screens.
- Read: Expo Router: layouts · src/shared/README.md
- Check yourself: why must code in `src/shared` never import from `src/modules`?

Concept 4.3 — Widgets: one module showing another's UI

- What to learn: a module can export a small self-contained widget (it loads its own data); another module places it on its screen.
- Read: Top-rated feature README
- See it in the app: `TopRatedCarousel` used in `home-screen.tsx`
- Check yourself: what does Home import, and from where?

Concept 4.4 — Screen states

- What to learn: every data screen handles 4 states: loading, error (with retry), empty, data.
- Read: Feature reference ("How it works")
- See it in the app: `src/shared/components/screen-states.tsx`
- Check yourself: what does the product list show when a search finds nothing?

Concept 4.5 — Accessibility

- What to learn: screen readers (VoiceOver, TalkBack) read `accessibilityLabel`; buttons need `accessibilityRole="button"`; touch targets should be big enough.
- Read: React Native: accessibility
- Check yourself: turn on VoiceOver or TalkBack and try the Products tab.

Concept 4.6 — Carts API

- What to learn: what the carts endpoints return (products, quantities, totals).
- Read: DummyJSON carts · open `https://dummyjson.com/carts/user/1` in your browser
- Check yourself: what is the difference between `total` and `discountedTotal`?

Tasks this week

Task	Name	Type	Size	Uses concepts
4.1	Offline banner	UI + Functional	S	4.1, 4.2
4.2	"Team" widget on Home	UI	M	3.1, 4.3
4.3	Cart tab (mini project, part 1)	UI + Functional	L	3.2, 3.4, 4.6 (on your own)
4.4	Cart summary and states (part 2)	UI	M	4.4, 4.5

Task 4.1 — Offline banner

- **Type:** UI + Functional · **Size:** S
- **Objective:** tell users clearly when they're offline, so they know why data isn't updating.
- **What the user sees:** when the phone goes offline, a thin bar appears at the top of every screen (below the status bar): a `WifiOff` icon and "You're offline. Showing saved data." When the connection is back, the bar disappears by itself. Screens that were already loaded keep showing their data.
- **Files:** `src/shared/components/offline-banner.tsx` (new) · `src/app/_layout.tsx` · `src/shared/sandbox/` (add a demo).
- **What to do:**
  1. Create `offline-banner.tsx`. Call `useNetworkState()` from `expo-network`. If `isConnected === false` (or `isInternetReachable === false`), render the bar; otherwise return `null`.
  2. Style: `flex-row items-center justify-center gap-2 bg-muted px-4 py-2`, text `text-muted-foreground text-sm`. Respect the safe area with `useSafeAreaInsets()`.
  3. Add `accessibilityRole="alert"` and `accessibilityLiveRegion="polite"` so screen readers announce it.
  4. Render `<OfflineBanner />` in `src/app/_layout.tsx` above the `Stack`.
  5. Add an always-visible preview of the bar to the UI Sandbox (shared components go there).
- **Done when:**
  - ☐ Airplane mode on → the bar shows on every screen; off → it disappears.
  - ☐ The bar never covers the status bar or the screen header.

Task 4.2 — "Team" widget on Home

- **Type:** UI · **Size:** M
- **Objective:** show the team on the Home dashboard, using only the users module's public API.
- **What the user sees:** on Home, a section titled "Team" with 5 round user photos side by side (slightly overlapping), a "+N" circle for the rest, and a "See all" link on the right that opens the Users tab. Tapping a photo opens that user's profile.
- **Files:** `src/modules/users/features/team-row/` (new: `team-row.tsx`, `index.ts`, `README.md`) · `src/modules/users/index.ts` · `src/modules/home/features/dashboard/home-screen.tsx`.
- **What to do:**
  1. Create `team-row.tsx` with a `TeamRow` component that loads its own data with `useUserList()` and takes the first 5 users. Look at `TopRatedCarousel` for the pattern.
  2. Header: `flex-row justify-between items-center` with "Team" (`font-semibold`) and a "See all" `Button variant="link"` that calls `router.push('/users')`.

3. Avatars: `flex-row`, each `Avatar` with `-ml-3` (except the first) and `border-2 border-background` so they overlap neatly. Each avatar opens `usersRoutes.detail(user.id)`.
4. While loading, show 5 grey circles (`bg-muted`); on error, hide the section.
5. Export `TeamRow` from `src/modules/users/index.ts` and place it in `home-screen.tsx`, importing only from `@/modules/users`.

- **Done when:**
  - [ ] Home shows the Team row; "See all" and each photo navigate correctly.
  - [ ] `npm run lint` passes (Home doesn't import any users internals).

Task 4.3 — Cart tab (mini project, part 1)

- **Type:** UI + Functional · **Size:** L · **Do this one on your own:** it's what Weeks 1–3 prepared you for. The steps are short on purpose.
- **Objective:** let the user see what's in their shopping cart.
- **What the user sees:** a new "Cart" tab (shopping cart icon). It lists each product in user 1's cart: thumbnail, title, "Qty: 2", the line price, and the discounted line price if there is a discount. Tapping a line opens that product's detail screen.
- **Files:** a new `src/modules/carts/` module · `src/app/(tabs)/cart.tsx` · `src/app/(tabs)/_layout.tsx`.
- **What to do:**
  1. Follow ADDING-A-MODULE.md with "carts" instead of "users": endpoint `GET /carts/user/{userId}`, schema (cart, cart line), service, keys, `useUserCart(userId)`. Note the API returns `{ carts: [...] }`: use the first cart.
  2. Write 2 service tests (cart loads, 404 throws).
  3. Create `features/cart-overview/` with the screen and a `cart-line.tsx` row component.
  4. Add the route `src/app/(tabs)/cart.tsx` and a tab with the `ShoppingCart` icon.
  5. Open a product with `catalogRoutes.detail(id)` imported from `@/modules/catalog`.
- **Done when:**
  - [ ] The Cart tab lists the products in user 1's cart with quantities and prices.
  - [ ] Tapping a line opens the product. `npm test` passes.

Task 4.4 — Cart summary and screen states (mini project, part 2)

- **Type:** UI · **Size:** M
- **Objective:** make the cart screen complete, accessible and production-ready.
- **What the user sees:** a summary card pinned to the bottom of the Cart tab: "Items" (total quantity), "Subtotal" (`total`, struck through if there is a discount), "You save" (difference, in `text-success`) and "Total" (`discountedTotal`, large and bold). While loading, a spinner; on error, a message with "Try again"; with an empty cart, an icon and "Your cart is empty" with a "Browse products" button that opens the Products tab.
- **Files:** `src/modules/carts/features/cart-overview/` · `src/modules/carts/README.md` · `src/modules/carts/components/carts-showcase.tsx` (new, optional).
- **What to do:**
  1. Create a `CartSummary` component (props: `totalQuantity`, `total`, `discountedTotal`) using the card style and the label/value rows from Task 1.3. Format money with `formatPrice` from `@/shared/lib/format`.
  2. Place it at the bottom of the screen, above the tab bar, with a top border.

3. Add `LoadingState`, `ErrorState` (with retry) and `EmptyState` (copy how `product-list-screen.tsx` does it).
4. Accessibility: each cart line gets an `accessibilityLabel` like "iPhone 15, quantity 2, 1,998 dollars"; the summary reads as one sentence.
5. Write the `README.md` for the carts module and each feature.

- **Done when:**
  - [ ] The summary numbers match the API.
  - [ ] All four states work. Every DummyJSON user has a cart, so test "empty" by temporarily rendering the screen with an empty `products` array, then undo it.
  - [ ] Light and dark mode look right, and `npm run validate` passes.

End-of-week check

- [ ] All 16 PRs open · `PROGRESS.md` complete · `npm run validate` passes.

After Week 4: Look back

1. Record a **2-minute screen recording** of your app showing everything you added.
2. Rate yourself from 1 (not at all) to 5 (confident) in `PROGRESS.md`:
    - I can build a screen with the app's components and theme colors. (UI)
    - I can add a route and navigate to it.
    - I can load data with a schema, a service and a React Query hook. (Functional)
    - I can change data with a mutation and see the screen update. (Functional)
    - I can build a form with validation.
    - I can create a new module by following the recipe.
    - I can get unstuck on my own.

`PROGRESS.md` template

```
# Progress — <your name>

## Learning

| Week | Concepts read (yes / partly) | My "Check yourself" answers (short) |
| ---- | ---------------------------- | ----------------------------------- |
| 1    |                              |                                     |
| 2    |                              |                                     |
| 3    |                              |                                     |
| 4    |                              |                                     |

## Tasks

| Task | Name                | Type            | Status (done / partly / skipped) | PR
link | Hours | Blockers |
| ---- | ------------------- | --------------- | -------------------------------- | ------
- | ----- | -------- |
| 1.1  | Support row         | UI + Functional |                                  |
|       |          |
| 1.2  | Rating stars        | UI              |                                  |
|       |          |
| 1.3  | Details card        | UI              |                                  |
|       |          |
| 1.4  | Reviews screen      | UI + Functional |                                  |
|       |          |
| 2.1  | Sort by price       | Functional      |                                  |
|       |          |
| 2.2  | Out-of-stock action | Functional      |                                  |
|       |          |
| 2.3  | Category screen     | UI + Functional |                                  |
|       |          |
| 2.4  | Discount field      | Functional      |                                  |
|       |          |
| 3.1  | Users data layer    | Functional      |                                  |
|       |          |
```

```
| 3.2  | Users tab           | UI              |                                  |
|      |          |
| 3.3  | User profile        | UI + Functional |                                  |
|      |          |
| 3.4  | Login               | UI + Functional |                                  |
|      |          |
| 4.1  | Offline banner      | UI + Functional |                                  |
|      |          |
| 4.2  | Home Team widget    | UI              |                                  |
|      |          |
| 4.3  | Cart tab            | UI + Functional |                                  |
|      |          |
| 4.4  | Cart summary        | UI              |                                  |
|      |          |
```

For the team lead: tracking progress

No teaching needed. Once a week, look at each trainee's fork:

- **Task PRs:** 4 per week is on track. Check each PR against the task's "Done when" list.
- **`PROGRESS.md`:** concepts read, "Check yourself" answers, hours and blockers.
- **UI tasks:** ask for a light and a dark screenshot in the PR.
- **Functional tasks:** check the error case and the tests.

Trainee	Week 1	Week 2	Week 3	Week 4	Self-check avg	Still active?
	/4	/4	/4	/4	/5	yes / no

Good signs: a PR for every task, `npm run validate` passing, short honest answers in "Check yourself", and blockers written down instead of silence.

After the 4 weeks

Next topics: protected routes and token refresh, optimistic updates, component tests, performance, development builds and EAS (publishing), push notifications.

Links: Expo tutorial · React Native docs · TanStack Query · NativeWind · React Native Reusables